



Nombre y Apellido:

Legajo:

Examen Parcial 2

1. Definimos el TAD `OrderedSet` (conjunto donde cada par de elementos puede compararse con una relación de orden) con las siguientes operaciones:

```
tad OrderedSet (A : Ordered Set) where
import Maybe, Bool
empty    : OrderedSet A          -- construye un conjunto vacío
insert   : A → OrderedSet A → OrderedSet A -- agrega un elemento al conjunto ordenado
find     : A → OrderedSet A → Bool  -- busca un elemento en un conjunto ordenado
minimum  : OrderedSet A → Maybe A   -- devuelve el menor elemento en un conjunto ordenado
delete   : A → OrderedSet A → OrderedSet A -- elimina un elemento del conjunto
ceiling  : A → OrderedSet A → Maybe A -- devuelve el menor elemento del conjunto mayor o
                                         -- igual al elemento dado
```

Dar una especificación algebraica del TAD.

2. Se representan secuencias mediante árboles binarios dados por el siguiente tipo de datos:

```
data Tree a = Leaf a | Node Int (Tree a) (Tree a)
```

donde se guarda la longitud de la secuencia en los nodos y el recorrido *inorder* del árbol da el orden de los elementos de la secuencia.

a) Definir en Haskell de manera eficiente la función `mapReduceIndex :: (Int → a → b) → (b → b → b) → Tree a → b`, que dados una función $f :: \text{Int} \rightarrow a \rightarrow b$, un operador binario asociativo $op :: b \rightarrow b \rightarrow b$ y una secuencia s , computa el resultado de aplicar el operador op sobre los elementos de la secuencia que resulta de aplicar la función f a cada elemento de s usando como primer argumento su índice en s , en el orden dado por el recorrido *inorder*.

Por ejemplo,

$$\text{mapReduceIndex } f \text{ op } \langle x_0, x_1, x_2 \rangle = ((f \ 0 \ x_0) \text{ 'op' } (f \ 1 \ x_1)) \text{ 'op' } (f \ 2 \ x_2)$$

Definir `mapReduceIndex` con profundidad en $O(h)$, donde h es la altura del árbol y f y op son de costo constante.

Plantee las recurrencias para el trabajo y la profundidad para el caso en que f y op son de costo constante. No hace falta resolverlas.

b) Usando la función `mapReduceIndex`, defina una función `sumEven :: Tree Int → Int` que compute la suma de los números en las posiciones pares de una secuencia.

Por ejemplo,

$$\text{sumEven } \langle 7, 1, 5, 7, 2 \rangle = 14$$

3. Para realizar operaciones de compra y venta de dólares se definieron los siguientes tipos de datos:

```
type Pesos = Float
type Dolares = Float
type Cotizacion = Float
data Transaccion = Depositar Pesos | Comprar Int Cotizacion | Vender Int Cotizacion
```

donde Depositar x representa la operación “depositar x pesos”, Comprar n c , “comprar n dólares con costo c pesos cada uno” y Vender n c , “vender n dólares con la cotización c ”.

La siguiente función calcula el saldo en pesos y dólares luego de una transacción:

```
eval :: Transaccion -> (Pesos, Dolares)
eval (Depositar pesos) = (pesos, 0)
eval (Comprar n cot) = let x = fromIntegral n in (-x * cot, x)
eval (Vender n cot) = let x = fromIntegral n in (x * cot, -x)
```

Diremos que una secuencia de transacciones es correcta si al evaluar cada transacción en el orden en que ocurren en la secuencia todas pueden realizarse. Por ejemplo, transOk es correcta, mientras que transFail1 y transFail2 no, ya que la 2° transacción no puede realizarse en transFail1 (no pueden venderse dólares sin haberlos comprado antes) ni en transFail2 (no se pueden comprar dólares sin un saldo suficiente en pesos).

```
transOk = [Depositar 100000, Comprar 100 1000, Vender 20 500, Comprar 5 1000]
transFail1 = [Depositar 100000, Vender 20 500, Comprar 100 1000, Comprar 5 1000]
transFail2 = [Depositar 100, Comprar 100 1000]
```

Usando las operaciones del TAD Secuencias, definir una función transaccionOk, que dada una secuencia de transacciones, determine si es correcta. Definir esta función con profundidad en $O(\lg n)$ y trabajo en $O(n)$, siendo n el largo de la secuencia.